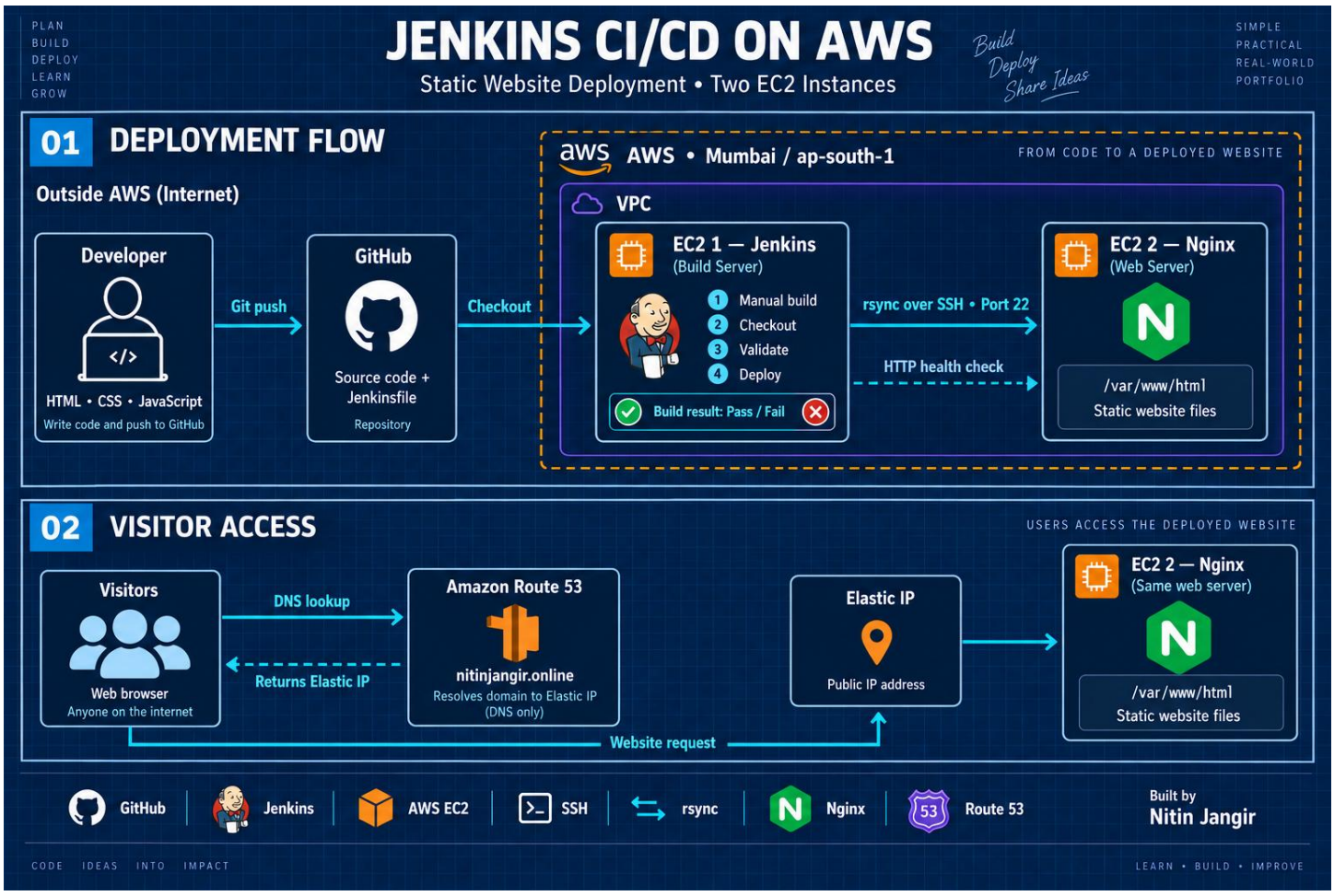
AWS-Based CI/CD Pipeline Using Jenkins

A static website deployment project built using Jenkins, GitHub, Amazon EC2, Nginx, SSH, and rsync. The project uses two EC2 instances to separate the Jenkins server from the web server, with an Elastic IP and Amazon Route 53 for domain access.

Technologies and AWS Services Used :-

- **Amazon EC2** – Hosts the Jenkins server and Nginx web server.
- **Jenkins** – Executes the website deployment pipeline.
- **Git and GitHub** – Manage source code, commits, and the Jenkinsfile.
- **Nginx** – Serves the deployed website.
- **SSH & rsync** – Provides remote access between the servers & transfers updated website files.
- **Elastic IP** – Provides a stable public address for the web server.
- **Amazon Route 53** – Maps the custom domain to the web server.
- **HTML, CSS, and JavaScript** – Build the website interface.

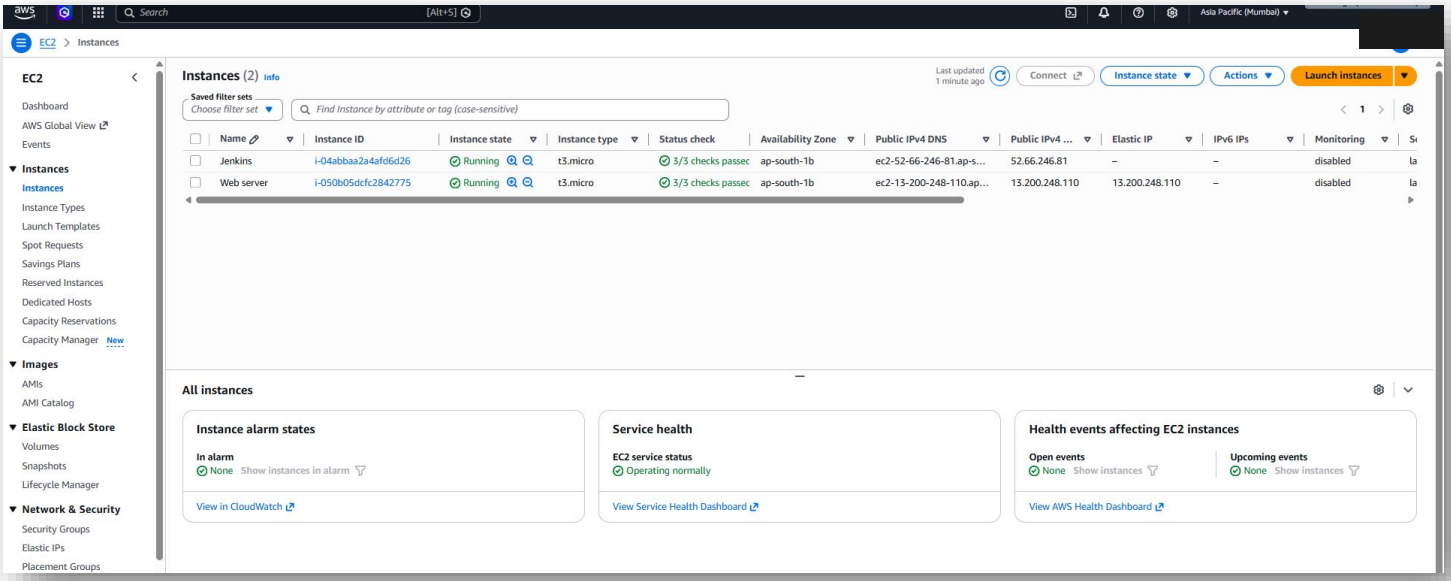
Architecture :-



The project uses two EC2 instances: a Jenkins server for pipeline execution and an Nginx server for website hosting. Jenkins retrieves the website source code from GitHub, validates the required files, and deploys them to the web server using rsync over SSH.

After deployment, an HTTP health check verifies that the website responds. Visitors access the website through the custom domain, which Route 53 resolves to the web server's Elastic IP.

EC2 Configuration :-

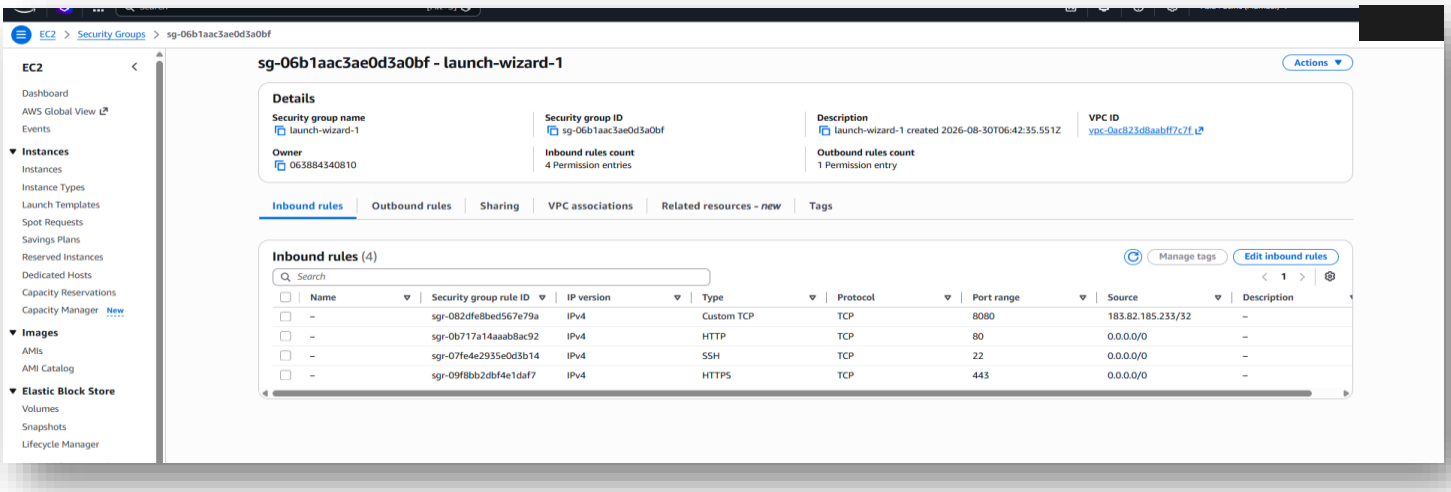


Two EC2 instances were configured in the AWS Mumbai region:

- **Jenkins Server:** Runs Jenkins and executes the deployment pipeline.
- **Web Server:** Runs Nginx and hosts the website files.

The screenshot shows both instances running with their status checks passing.

Security Group Configuration :-



Security group rules control access to SSH, the Jenkins dashboard, and web traffic. The displayed configuration restricts Jenkins port 8080 to a self IP address and includes rules for ports 22, 80, and 443.

The captured configuration allows SSH from all IPv4 addresses. Restricting SSH access is a remaining security improvement.

SSH Connectivity Between Servers:-

```
root@jenkins:~# echo "SSH TEST: Jenkins server to web server"

ssh -i /var/lib/jenkins/.ssh/id_ed25519 \
ubuntu@172.31.5.150 \
'echo "SSH CONNECTION SUCCESSFUL"; hostname; hostname -I; whoami; systemctl is-active nginx'
SSH TEST: Jenkins server to web server
SSH CONNECTION SUCCESSFUL
webserver
172.31.5.150
ubuntu
active
root@jenkins:~#
```

SSH connectivity was tested from the Jenkins server to the web server using an SSH key. The remote command returned the web server's hostname, private IP address, and username, and confirmed that Nginx was active.

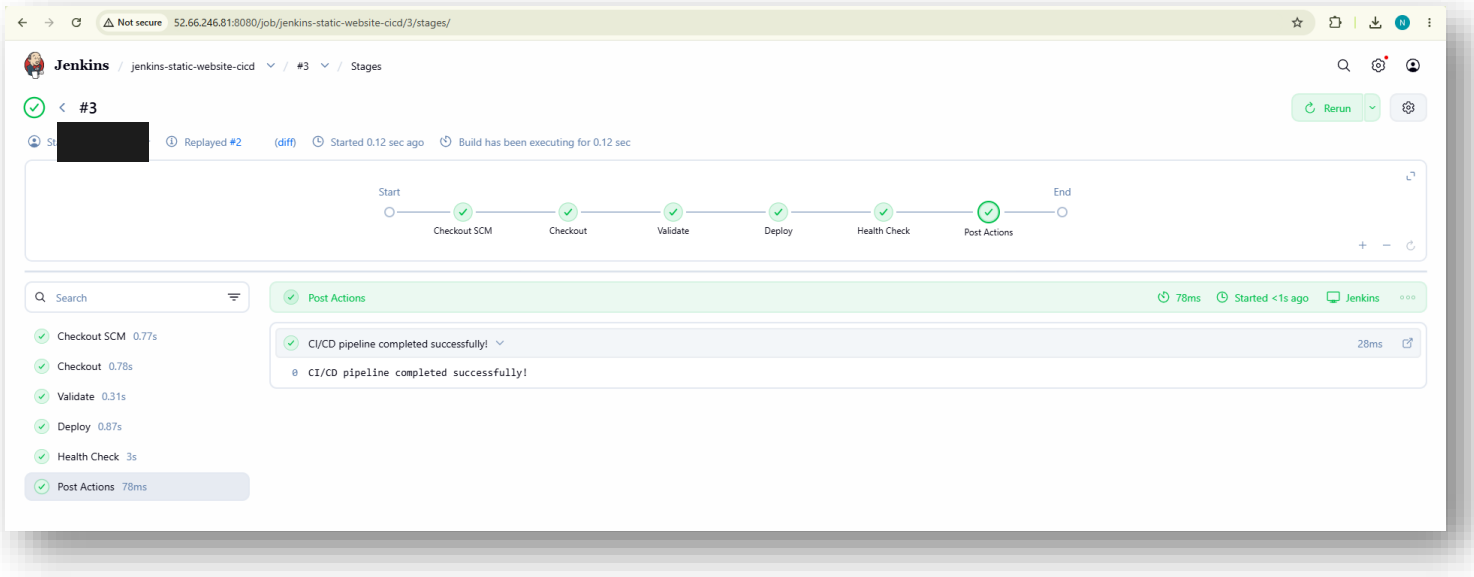
This verifies the server-to-server connection used for deployment.

GitHub Integration and Version Control :-

The website source code and Jenkinsfile were maintained in GitHub. Jenkins retrieved code from the main branch and recorded the commit associated with each build.

The screenshot shows build #9 linked to the commit named "branding name changes," making it possible to identify the source revision used by that build.

Jenkins Pipeline Execution :-



The Jenkins pipeline performs the following steps:

1. **Checkout:** Retrieves website source code from GitHub.
2. **Validate:** Checks the required website files.
3. **Deploy:** Transfers the files to the Nginx server.
4. **Health Check:** Checks the website's HTTP response.
5. **Post Actions:** Reports the pipeline result.

The screenshot shows all displayed stages completing successfully. The demonstrated builds were manually started, after which Jenkins executed the configured steps automatically.

Build History :-



Jenkins maintains a history of pipeline runs, allowing previous builds and their results to be reviewed. The screenshot shows multiple successful builds recorded during development and website updates.

Build Duration :-

